

Bash Scripting

1. Permission rights
2. What are bash Scripts?
3. Hello World
4. Comment, Comment, Comment
5. Variables
6. External inputs
7. Arithmetic in Bash
8. if statements
9. Loops
10. Functions
11. Nested if statements

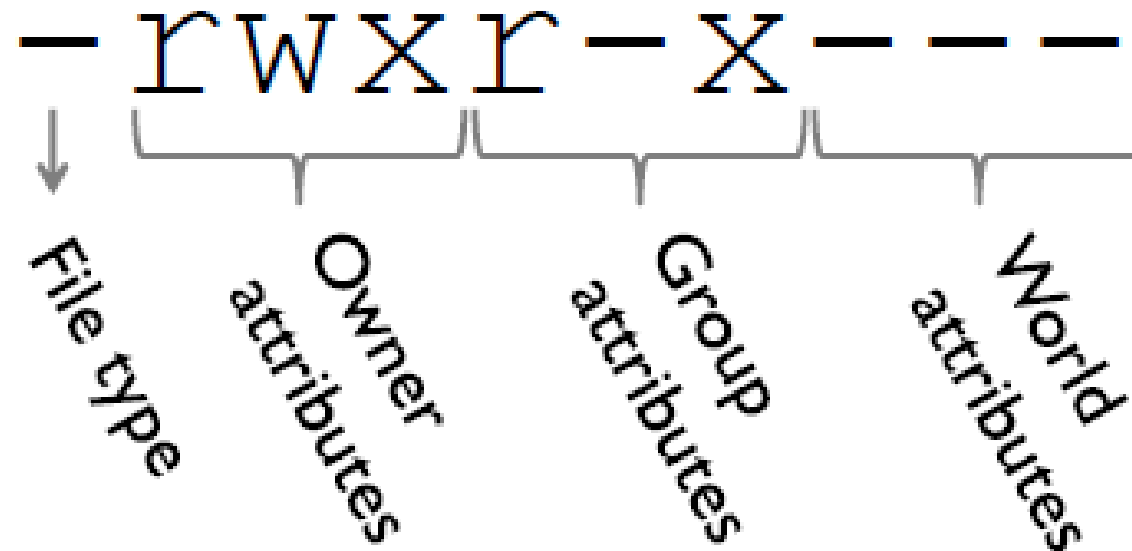
Permission rights

- User can be real persons
- User can be objects of the system
- Each user on a Linux system gets an internal unique user identification number (UID)
- User can be hoarded into groups
- Groups also have an unique group identification number (GID) in the system
- Groups allow to share permissions between multiple users

- Linux systems are environments which can be used by multiple users, who are able to create files, run programs and share data
- Files and directories therefore always have two owners
 - ▶ The user who created the file/directory
 - ▶ The group, consisting of at least one user, for which the file/directory was created

- Files and directories have three types of permission rights, that form so-called triplets:
 - ▶ Read (r)
 - ▶ Write (w)
 - ▶ Execute (x)
- Each file and directory has three levels of permission rights:
 - ▶ User or Owner
 - ▶ Group
 - ▶ Others or “World”

- Each triplet defines the permission rights for one level
- In the example below:
 - ▶ the user has read/write and execution rights
 - ▶ the group has read and execution rights
 - ▶ the others don't have any rights



- From left to right:

- ▶ Permission rights
- ▶ Hard links
- ▶ Owner
- ▶ Group
- ▶ File size in bytes
- ▶ Modification date
- ▶ Name

```
cknorr@DESKTOP-813B69Q:~$ ls -l
total 8000
-rw-r--r-- 1 cknorr cknorr 457650 Aug  2  2022 1kim.pdb
-rw-r--r-- 1 cknorr cknorr  2298 May 16  2018 455373.gb
drwxr-xr-x 2 cknorr cknorr  4096 Oct  9 09:38 ABI_Files-Linux
-rw-r--r-- 1 cknorr cknorr 1945638 Jul 31 09:18 ABI_Files-Linux.zip
-rw-r--r-- 1 cknorr cknorr 122157 Mar 24  2022 Alpha_helix.png
drwxr-xr-x 3 cknorr cknorr  4096 Aug 30 13:38 Bash
drwxr-xr-x 3 cknorr cknorr  4096 Aug 29 15:03 Bash_New
-rw-r--r-- 1 cknorr cknorr    79 Jan 20  2023 Biol_Processes.txt
drwxr-xr-x 2 cknorr cknorr  4096 Aug  7 09:31 Biopython
-rw-r--r-- 1 cknorr cknorr    75 Jan 20  2023 Capitals.txt
-rw-r--r-- 1 cknorr cknorr    83 Jan 20  2023 Countries.txt
drwxr-xr-x 3 cknorr cknorr  4096 Aug  7 09:31 Databases
-rw-r--r-- 1 cknorr cknorr   122 Jan 20  2023 Fields_Biology.txt
-rw-r--r-- 1 cknorr cknorr   417 Jan 20  2023 File_Generator.py
-rw-r--r-- 1 cknorr cknorr   323 Jan 20  2023 File_Types.txt
drwxr-xr-x 3 cknorr cknorr  4096 Aug  7 09:24 GUI
```


- The command `chmod` allows you to change permission rights for files you own
- This command takes two arguments
 1. The rights you want to modify
 2. The file you want to change the permission rights for
- The first argument of `chmod` is composed out of three different parts (next slide) and uses the following general syntax:
 - ▶ L1oL2

1. The level for which you want to change the permission rights (L1)

- ▶ u=user
- ▶ g=group
- ▶ o=other or world
- ▶ a=all three of the above

2. Do you want to add or remove the permission rights (o)

- ▶ +=adding
- ▶ -=removing

3. The type of permission rights you want to change (L2)

- ▶ r=reading
- ▶ w=writing
- ▶ x=executing

■ Examples:

- ▶ u+x would grant execution rights for the user
- ▶ o-r would remove reading rights for others

- To give a group reading rights for a file called myfile.txt you would use:

```
chmod g+r myfile.txt
```

- To remove all permission rights for a file called myfile.txt for other you would use:

```
chmod o-rwx myfile.txt
```

- As you can see with the last example you're able to modify multiple types of permission rights at once
- In a similar manner you're able to modify multiple levels of permission rights at once as well:

```
chmod go-r magic.txt
```

- The command above remove reading rights for the group and others

What are bash scripts?

- A Bash script is a plain text file which contains a series of bash commands
- Anything you can run normally on the command line can be put into a script and it will do exactly the same thing
- Similarly, anything you can put into a script can also be run normally on the command line and it will do exactly the same thing
- You don't need to change anything
- Just type the commands as you would normally and they will behave as they would normally
- It's just that instead of typing them at the command line we are now entering them into a plain text file

- per convention Bash scripts are files with the ending `.sh`, even though we don't need any extensions under Linux
- All Bash scripts have to be made executable before running them by using the command:
 - `chmod u+x example.sh`
- You have to define an interpreter, thus all your scripts should start with:
 - `#!/bin/bash`
- Scripts can be started by using:
 - `./example.sh`

- Bash Scripts can contain the same programming constructs we saw in action in Python
- That means we can define variables, run if statements, create loops and define our own functions inside these scripts
- They are just programs that we write in a different programming language, bash in our case, and therefore follow similar rules as python programs
- During this lesson we will talk about how to set up the same constructs as in Python with the help of bash scripts

- As you will see bash scripts can be a bit more complicated than Python scripts
- Bash scripts also have less flexibility and functionality than Python or R
- Still there are companies that use bash scripts up until today for certain tasks
 - Never change a running system
- Furthermore bash scripts can be especially useful for creating directory systems or for copying / moving lots of files

Hello World

- Just like with Python we will start with a simple bash script called “Hello World”, that simply prints out the statement “Hello World” into the terminal
- The complete script consists of two lines of code:

```
#!/bin/bash
```

```
echo Hello World!
```

- The first line is the shebang, more about this on later slides
- The second line is the command echo, which works similar to print since it allows us to print something out into our terminal
- Unlike with print, we don’t need any brackets, instead everything written after echo, here “Hello World!” is printed out in the terminal

- I saved the file we just saw under the name `01_hello.sh`, so that I know this is a bash script
- Now I want to run this script, to do so I have to go over three steps:
 1. Open the shell and go to the directory where your script is saved
 2. Make the script executable with the following command: `chmod u+x 01_hello.sh`
 3. Start the script with the command: `./01_hello.sh`
- Now you will see the print out `Hello World!` in your terminal and that means the script worked just fine
- If you want to start your file again you need to repeat ONLY the third step
- For new files however you need to repeat ALL steps first

- As you saw we need to add ./ before the name of our script to be able to execute it in our terminal
- Normal Linux commands are executed just with the name of the command, `ls` for example, but here we have to do something else
- The reason for this is, that Bash, when trying to execute a command, only looks in specific directories and doesn't consider sub directories or your current directory in this process
- Therefore we need to tell Bash where it can find our script so that it can be executed
- The ./ refers thereby to our current working directory, the directory our Shell is in

- Since we need to specify the place where we can find our script we are able to work with relative and absolute paths
- The command `./01_hello.sh` is a relative path, starting at our current working directory, represented by the `.` and in there we have the file `01_hello.sh`
- Another way would be using absolute paths like:
 - ▶ `/home/username/Bash/01_hello.sh`
- Like this it doesn't matter where our shell is located and we could Bash scripts from other directories
- However for now please stick to the version with the relative path and maneuvering to the corresponding directory where your script is beforehand

- We saw that the first line of our script, and to be honest the first line of all our scripts should be like this, contains the following:
 - ▶ `#!/bin/bash`
- In this line the `#!` is called shebang and following it is the absolute path to the interpreter (or program) that should be used to run (or interpret) the rest of the lines in the text file
- For our scripts that is the path to Bash, `/bin/bash`
- There are other scripts that have other interpreters, which we could name here as well

- There are several very important rules you have to follow:
 1. The shebang ALWAYS has to be the very first line, you can't even have an empty line before it
 2. There are NO spaces allowed between the #, the ! and the path
 3. ALWAYS include the shebang in your scripts to avoid any unnecessary troubles

- Use a text editor and create your first shell script
- The script should include the following commands
 - ▶ `ls`
 - ▶ `cal`
 - ▶ `date`
 - ▶ `pwd`
- Don't forget the shebang in the first line
- Save the file and make it executable with `chmod` afterwards
- Try to run this file

Comment, Comment, Comment

- Regardless of which language you use in the future, comments in your code are essential for understanding your scripts even after a long time
- In bash scripts you can add comments by using “#” and typing your comment afterwards
- Bash only allows single line comments, that means you need to start each line that contains comments with a #
- You can follow the same rules for comments you learned under Python, since the formatting follows the same rules

- Take your script from exercise 14 and add comments to the lines already present in the script
- Add at least two of the three tasks below:
 - ▶ Clear the terminal
 - ▶ Let the script greet the current user
 - ▶ Let the script display the home directory (echo \$HOME)
- Don't forget to comment the new commands as well

Variables

- You should already be familiar with the term variable
- As a reminder: We use variables to save certain values, numbers or texts, under a certain name, which allows us to reuse the same value over and over again without the need of changing it at each occurrence
- Of course Bash scripts which are also just simple programming scripts are able to use variables in a similar manner

- The definition of variables in Bash is very similar to the definition of variables in python, as it follows the same principle of `variablename=variablevalue`
- In Bash we can define simple variables as follows:

```
myname=Christoph
```

```
myage=35
```

```
myhours=40.5
```

```
myfullname="Christoph Knorr"
```

- As you can see we can define string (myname), integer (myage) and float (myhours) variables in a very easy manner

- When defining variables in Bash scripts we have to follow some essential rules:
 1. We can't have any spaces between the variablename, the equal sign and the variablevalue
 2. Variablenames can NOT contain spaces
 3. If our string variables have more than one word we need to use quotation marks („ „) around the variablevalue

- Now that we defined our own variables we want to work with them
- To do so we have to read out the value that is saved in our variables
- Unfortunately this is not as easy as in Python, where we only used to variable name to get access to the value
- Instead we need to add the symbol \$ before the name of our variable to read it out:

```
echo $myname
```

```
echo $myage
```

```
echo $myhours
```

```
echo $myfullname
```


- Create a bash script that defines six different variables:
 - ▶ two integer variables
 - ▶ two floating point variables
 - ▶ two string variables, one with more than one word
- Print them out using the command echo
- Try to create echo statements with multiple variables as well

- You saw how to define your own variables in bash scripts with the help of values you specified
- We can also use the outputs of commands as values for our variables
- This process is called command substitution
- To do so we have to define a variable similar to what we did beforehand, but now the right side of our equal sign will look different
- The general syntax is:
 - `variablename=$( command )`

- Let's say we want to save the output of the command `ls` in a variable
- We can do this by using the following code:

```
list1=$( ls )
```

- What are we doing here?
- First we define the name of our variable (`list1`)
- Than we tell bash we want to use the output of a command by using the `$` sign
- Now we have to specify the command we want to use inside the brackets, here by simply writing `ls`
- We can of course add options to the command as well:

```
list2=$( ls -la )
```

- When playing about with command substitution it's a good idea to test your output rather than just assuming it will behave in a certain way
- The easiest way to do that is simply to echo the variable and see what has happened
- You can then remove the echo command once you are happy
- In general we can use the output of any linux command (or combinations of them using pipes) as value for our variables
- Just keep in mind that, the outputs can change, especially when working with commands like `ls`, since their outputs depend on the files and directories you have in your current working directory

- Create a bash script that defines variables by using the command substitution for the following commands:
 - ▶ List the contents of your current working directory in reversed order
 - ▶ Search for files (using find) with the extension .sh
 - ▶ The execution of your bash script from Exercise 16

- Aside of variables we define on our own, we also have access to some pre-defined variables:

\$0	➔	Contains the name of our bash script
\$1 – \$9	➔	Values handed over to our script as arguments (see next topic)
\$#	➔	The number of arguments handed over to our script
\$@	➔	All arguments handed over to our script
\$USER	➔	the username of the user running the script (has not to be the one who wrote the script)
\$SECONDS	➔	Seconds since the script was started
\$RANDOM	➔	A random number that changes each time it's referenced

External inputs

- There are different ways of handing over values to a bash script via inputs
- Here I want to show you only the use of command line arguments:
 - ▶ We add values for our variables after the name of our script when we want to execute it

- When starting a bash script we can add as many arguments after the name as we want by using the following syntax:
 - `./script argument1 argument2 argument3 ...`
- The maximal amount of arguments we can hand over to a bash script depends on the system your working on, but in most cases the limit is somewhat around 1 or 2 million

- Inside our script we can access the handed over arguments by using pre-defined variables named \$1, \$2 and so on for single numbers as well as \${10}, \${11} and so on for multi-digit numbers
- Which argument is handed over to which variable is defined by the order of the arguments you write in the commandline
- Starting a script like this:

```
./commandline.sh apple orange banana
```

- Would lead to \$1 getting the value apple, \$2 getting the value orange and \$3 getting the value banana for example

- Imagine a script (args.sh) like this:

```
#!/bin/bash
```

```
echo $1
```

```
echo $2
```

```
echo $3
```

- Here we want to print out the values of the first three arguments we hand over to our script

- We can now start the script like this:

- ./args.sh apple orange banana

- But what happens if we start it like this:

- ./args.sh apple orange

- In this case we don't have a third argument
- Now \$3 simply has no value, is left blank, BUT our script still works just fine

- We are able to use these handed over arguments in combination with other linux commands
- We could have a script (args.sh) that has the following code somewhere:

```
cp $1 $2
```

- Now we could start the script as follows:

```
./args.sh hello.sh hello_new.sh
```

- The script would then read:
 - cp hello.sh hello_new.sh
- And thus the script hello.sh would be copied into the script hello_new.sh

- Write a bash script that can create a directory with at least 6 subdirectories
- Use an external value in form of a command line argument handed over to the script at the start of it for the name of the main directory
- Use this command line argument only as name for the main directory

Arithmetic in Bash

- When working with bash scripts, just like with Python scripts, sometimes you need to use some form of arithmetics to be able to fulfill the tasks you need to do
- Here I want to show you two simple ways of running some very, very elemental mathematical operations using bash scripts
- These two methods are:
 - ▶ the built-in function `let`
 - ▶ the usage of double parentheses `(( ))`

- The command let is a built-in function that allows us to do simple mathematical calculations
- The general syntax is as follows:
 - let <arithmetic expression>
- For example we can define a variable num1 as sum of the numbers 5 and 4:

```
let num1=5+4
```

- Please be aware that we can't use spaces in these equations
- If you want to use spaces you need to write the complete expression in quotation marks:

```
let "num1 = 9 + 4"
```


- Of course we can use other mathematical operators with let as well:

```
let num2=25-17
```

```
let num3=3*8
```

```
let num4=250/5
```

- Furthermore we can increase the value of a variable by one with let as well:

```
let num1++
```

- For decreasing the value of a variable by one we can use:

```
let num2--
```

- Last but not least we can increase the value of a variable by a certain number:

```
let num1+=20
```

- Write a bash script that defines six variables by making use of the command `let` and some mathematical calculations
- The script should include each of the four basic mathematical operations
- Later on in the script do the following:
 - ▶ Increase the value of the first variable by 10
 - ▶ Decrease the value of the second variable by 15
 - ▶ Increase the value of the third variable by 1 four times in a row
- Print out the original values as well as the adjusted values using `echo`

- Instead of using the command let we can also work with double parentheses instead:
 - `$((expression))`
- If we take our example from earlier with num1 being the sum of 5 and 4 we can now use:
 - `num1=$((5+4))`
- When working with double parentheses spaces don't matter so we can also write something like this:
 - `num1=$(( 9 + 4 ))`

- Just keep in mind that you can NOT have space between the variable name, the equal sign, the \$ sign and the two opening parentheses ((
- Of course we can use other mathematical operators with double parentheses as well:

```
num2=$ ( (25-17) )
```

```
num3=$ ( (3*8) )
```

```
num4=$ ( (250/5) )
```

- Furthermore we can increase the value of a variable by one with let as well:

```
( (num1++) )
```

- For decreasing the value of a variable by one we can use:

```
( (num2--) )
```

- Last but not least we can increase the value of a variable by a certain number:

```
( (num1+=20) )
```

- Write a bash script that does the same calculations as the one from exercise 19
- This time however use double parenthesis (()) instead of let

- When working with these arithmetics we often work with variables as well, that means we want to add up or subtract the values of two variables from each other for example
- Of course bash scripts allow us to do so by using either let or double parentheses
- Imagine we have the following two variables num1 and num2:

```
num1=25
```

```
num2=12
```

- In a first example we want to add up the values of num1 and num2 using let:

```
let num3=num1+num2
```

- Here we add the values of num1 and num2 and assign the result to num3
- As you can see we don't need the \$ sign to access the values of our variables here

- In a second example we want to subtract the value of num2 from the value of num1 using double parenthesis:

```
num4=$((num1-num2))
```

- Again we can leave out the \$ to access the values of num1 and num2

- Write a bash script that defines two integer variables
- Define four additional integer variables as result of the four basic mathematical operations using the two first numbers:
 - ▶ number3 is the addition of number1 and number2
 - ▶ number4 is the subtraction of number2 from number1
 - ▶ number5 is the multiplication of number1 and number2
 - ▶ number6 is the division of number2 by number1
- Print out the values of all six variables
- You can either use let or double parenthesis for this exercise

if statements

- As we learned if statements allow us to execute only certain parts of our code depending on different conditions we can define
- Bash scripts allow us to create them as well, however the syntax differs from the syntax we learned for Python
- The general structure of an simple if statement, without else, looks like this:

```
if [[ <some test> ]]
```

```
then
```

```
    <commands>
```

```
fi
```

- Imagine we want to create a script where we want to compare a number the user hands over to the script as command line argument with the number 100
- We want our handed over number, the variable \$1 to be greater than 100
- So we can use the following if statement to achieve this:

```
if [[ $1 -gt 100 ]]
```

- In the example above `-gt` stands for “greater than” and thus we compare the value of the variable \$1 with the number 100 and if the value of \$1 is greater than 100 the statement becomes True

- Now we need to add commands to our if statement, that we want to execute when our statement becomes True
- For this example we just want a simple print out
- To do this we have to use the command then and afterwards we need to write down the commands that should be executed:

```
if [[ $1 -gt 100 ]]
```

```
then
```

```
    echo Hey that is a large number.
```

- Please be aware that ALL commands you want to execute as soon as your statement becomes True need to be indented, just like we saw in python

- Last but not least we need to close our if statement by adding the command `fi` at the end
- We ALWAYS need `fi` at the end of our if statements and it needs to have the same indentation as the if then construct it refers to:

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number.
fi
```

- In the example we saw that we used square brackets `[[ ]]` to formulate our comparison and in there we used the statement `–gt` to ask whether or not `$1` is greater than 100
- This is a reference to the command `test`, which we could use in our terminal as well
- There are many different operators available and you could look them up in the man page of the command `test`
- However on the next slide we will go over some of the more common ones

Expression	Example	Description
-n STRING	-n "Hello World"	the length of STRING is greater than zero
-z STRING	-z "Hello World"	the length of STRING is zero, i.e. it is empty
STRING1 = STRING2	Hello = World	STRING1 is equal to STRING2
STRING1 != STRING2	Hello != World	STRING1 is not equal to STRING2
INT1 -eq INT2	15 -eq 30	INT1 is equal to INT2
INT1 -gt INT2	15 -gt 30	INT1 is greater than INT2
INT1 -ge INT2	15 -ge 30	INT1 is greater than or equal to INT2
INT1 -lt INT2	15 -lt 30	INT1 is less than INT2
INT1 -le INT2	15 -le 30	INT1 is less than or equal to INT2
INT1 -ne INT2	15 -ne 30	INT1 is not equal to INT2
-d FILE	-d Hello	FILE exists and is a directory
-e FILE	-e hello.sh	FILE exists

- Write a bash script that defines four integer variables and two String variables
- Create six different if statements to compare your variables with each other
- Include simple echo commands in them to give back positive answers

- In python we learned that we can add an optional else statement to our if statements to include code that is executed as soon as the predicate of our if becomes False
- Again bash scripts allow us to do so as well
- We simply add the else after the commands that are part of the then command

```
if [[ <some test> ]]
then
    <commands>
else
    <other commands>
fi
```

- For example we could add an else statement to our simple if statement we created earlier
- We would just add else at the same indentation level as if and then and add some commands afterwards
- Please keep in mind the the fi command closes your whole construct and therefore comes AFTER the commands of the else statement

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number
else
    echo Such a small number
fi
```

- Similar to how we can add else statements to our if statements we are also able to chain together different checks using if elif else
- Again just like in Python as soon as one of our tests becomes True, the corresponding code is executed and everything else is ignored

```
if [[ <some test> ]]
then
    <commands>
elif [[ <some test> ]]
then
    <different commands>
else
    <other commands>
fi
```

- For example we could add an additional check to see whether or not our number is at least bigger than 10
- By simply adding an elif statement, including another then, before the else statement we can achieve such a task:

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number
elif [[ $1 -gt 10 ]]
then
    echo At least two digits
else
    echo Such a small number
fi
```

- Write a bash script that defines four different integer variables
- Create an if/elif/else statement that compares the first number with the remaining three numbers one by one using the following conditions
 - ▶ if number1 equal to number2
 - ▶ elif number1 bigger than number3
 - ▶ elif number1 smaller than number4
 - ▶ else
- Include simple echo commands inside each if/elif/else to print out answers

- Last but not least we are able to combine multiple checks in one if statement by using logical operators, just as we saw in Python
- For bash scripts we can NOT use the word and/or and need to use special characters for them instead:
 - && → represents AND
 - || → represents OR

- Write a Bash script that contains an if/elif/else statement and makes use of logical operators
- Run the following comparisons:
 - ▶ Number 1 = Number 2 and Number 3 = Number 4
 - ▶ Number 1 = Number 3 and Number 2 = Number 4
 - ▶ Number 1 = Number 4 or Number 2 = Number 3
- Include simple echo commands in each if/elif/else statement

Loops

- Just as we learned in Python we can also use loops in our bash scripts to repeat source code in an elegant way
- We have three types of loops available to us in bash scripts:
 - ▶ while loops
 - ▶ until loops
 - ▶ for loops
- While and for loops work identical to their counterparts in Python
- Until loops on the other hand are very similar to while loops in their general behavior but have a different logic

- Bash allows us to define lists of values in a relative simple manner
- We can do so by defining a name for our list and then hand over multiple values, space separated inside of quotation marks to it:
 - `names='Stan Kyle Cartman'`
 - `numbers='5 6 7 8 9'`
- Please be aware that you do NOT have any spaces between the list name, the equal sign, the first quotation mark and the first element of your list
- These are not lists in the sense of Python, where we could access each element separately instead Bash simply interprets this long string as something that contains multiple elements

- For loops do a specific task for each element in a given list or value range
- The general syntax is as follows:

```
for var in <list>  
do  
    <commands>  
done
```

- First we have the loop constructor with the command for
- Than we have to name our loopvariable (var) and of course we can freely choose the name here
- Afterwards we have to tell Bash over what the loop should iterate with the help of the keyword in and for example the name of a list afterwards

- Let's say we have a list of names:

```
names='Stan Kyle Cartman'
```

- Now we want to loop over this list and to do so we have to construct our loop first:

```
for name in $names
```

- This loop constructor now tells Bash that we want to loop over the list names, please be aware that we need the \$ sign here to read out the values of our list

- Now we need to tell Bash what we want to do for each element of our list
- We need to add the command `do` and then the commands we want to execute for each iteration of our loop:

```
for name in $names
```

```
do
```

```
    echo $name
```

- Here we just print out each name separately and again we need the `$` sign to read out the corresponding value

- Last but not least we need to close our for loop with the help of the command done:

```
for name in $names
do
    echo $name
done
```

- Now our loop is finished and when we run the script we will see that each of our three names is printed out separately in our terminal

- Of course we can include more than just one command in our loop
- Let's say we have a list of numbers as shown on the right side and want to add a constant value of 20 to each number
- In these cases we could use a loop as you can see on the right side and combine arithmetic calculations with the echo command inside our loop

```
numbers='5 6 7 8 9'
```

```
for num in $numbers
```

```
do
```

```
    ((num+=20))
```

```
    echo $num
```

```
done
```



```
for value in {1..5}
do
    echo $value
done
```

```
for value in {10..0}
do
    echo $value
done
```

- We can use for loops in combination with numerical (integer) ranges as well
- In Bash we specify such ranges with curly brackets { } and then write down the starting point (1 or 10) add two points (..) and then write the end point of our range (5 or 0)
- Now the loop iterates over these ranges with a stepsize of 1 (start and end point included)
- As you can see if the end point is smaller than the start point we decrease the values

```
for value in {10..0..2}
do
    echo $value
done
```

- Last but not least when working with ranges, we can specify a stepsize by simply adding it inside the curly brackets
- To do so we simply add two points .. and then the corresponding step size (2)

- Bash allows us to define ranges using curly brackets:
 - ▶ `{start..end}`
- Start and end point are included in the range and are separated by two points
- If the end point is smaller than the start point we get a decreasing range
- We can add a stepsize to our ranges:
 - ▶ `{start..end..step}`
- Now we increase / decrease the values in our range according to the step size instead of the default value of 1
- **ATTENTION:** Spaces are NOT allowed inside the curly brackets!!!

- Write a bash script that uses for loops to fulfill the following tasks:
 - ▶ print out all numbers from 20 to 34
 - ▶ print out only even numbers between 220 and 160 in decreasing order, you start at 220
 - ▶ print out multiples of 5 between 1000 and 1500

- Similar to Python, bash also allows us to create while loops that work with a condition and repeat command as long as the condition remains True:

```
while [[ <some test> ]]  
do  
    <commands>  
done
```

- The tests we can use when constructing such a while loop are the same as the ones we used for the if statements
- ATTENTION: Don't forget to include a way for your loop to end when working with while loops!!!

- A very simple example for while loops is to repeat a process as long as a variable called counter is less than or equal to a certain threshold
- We can construct such a while loop by defining the variable counter first:

```
counter=0
```

- Now we can construct our while loop with a simple test:

```
while [[ $counter -le 10 ]]
```

- This tells Bash to repeat the process afterwards as long as the value of the variable counter is less than or equal to 10, that means it can take on the values 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 and 10

- Now that we have the constructor of our loop we can create the body of it
- This process follows the same rules, that means we need the keyword `do`, the commands we want to execute (Don't forget a way to update the value of counter) and we need to close the loop using `done`:

```
while [[ $counter -le 10 ]]
do
    echo $counter
    ((counter++))
done
```

- The loop above simple prints out counter and than increases the value of it by 1

- While loops are used to repeat a task for as long as a certain condition stays True
- As soon as this condition becomes False the loop stops and we go on with the rest of our script

- Write a bash script that uses two different while loops to fulfill the following tasks:
 - ▶ print out all numbers from 30 to 20 in decreasing order
 - ▶ add a number to itself as long as the value of it is smaller than 10000

- The until loop is fairly similar to the while loop
- The difference is that it will execute the commands within it until the test becomes true
- General Syntax:

```
until [[ <some test> ]]  
do  
    <commands>  
done
```

- If we think back to our example for the while loop, we could achieve the same thing with an until loop as well
- The solution would look like the one we see on the right side
- Please be aware that the condition is different now
- Instead of `$counter -le 10` we need to use `$counter -gt 10` to make sure the condition stays False until we reach the threshold

```
counter=1

until [[ $counter -gt 10 ]]
do
    echo $counter
    ((counter++))
done
```

- Take your script from Exercise 26 and rewrite it to use until loops instead of while loops

- We don't necessarily
- The while loop would be able to handle every scenario
- Sometimes, however, it just makes it a little easier to read if we phrase it with until rather than while
- So by having both while and until we can pick whichever one makes the most sense to us and as a result, end up with code that is easier for us to understand when we read it

Functions

- As we learned in Python functions are a good way to reuse code over and over again, while on the same time improving readability and saving space
- In bash we can also define such functions and they follow similar rules as the ones in Python
- Down below you see the two ways of defining functions in bash:

```
function_name () {  
    <commands>  
}
```

```
function function_name {  
    <commands>  
}
```

- Here we will discuss only the absolute basics when working with functions in bash
- We start with a very simple function that just contains a small print out and is called `print_something`
- We can define a function just like this:

```
print_something () {  
    echo Hello I am a function  
}
```

- This is more than enough for defining a simple function that doesn't use any arguments

- Just like with Python we should define our functions at the top of the script before we do anything else, so that we can use them afterwards
- The call, that means executing a self defined function than becomes relatively easy afterwards
- We simply write down the name of our function as we would with any other command and it's executed on the spot:

```
print_something
```

```
print_something
```

- Of course defining functions that do NOT use any arguments is not very useful
- Instead we want to create functions that can work with different arguments we hand over to them
- To do so we need to adjust the definition of our function a bit:

```
print_something () {  
    echo Hello $1  
}
```

- As you can see I added the variable \$1 to my function and we already saw it in action when talking about handing over arguments to our script

- It follows the same logic here, that means the first argument we hand over is assigned as value to \$1 and so on
- Now we can adjust the call back as well:

```
print_something Mars
```

```
print_something Jupiter
```

```
print_something
```

- As you can see handing over arguments to our function is easily done by simply writing the corresponding values after the function name (space separated)
- Even if we don't hand over any arguments our function still works, \$1 is just left empty in these cases

- Write a bash script that defines four different functions
- Each of these functions should take two integer values and calculate the result for one of the four basic mathematical operations (+, -, *, /)
- Print out the result of the calculations inside the function

Nested if-statements

- When talking about if statements in python we learned that we can nest them into each other to create logical rows of checks
- Of course we can do the same with bash scripts as well
- Let's say we want to adjust our earlier example by checking whether or not the value of \$1 is less than 1000 after we made sure it's greater than 100
- We start with our statement from before:

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number.
fi
```

- Now we can add another level of if statements inside our first if:

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number.
    if [[ $1 -lt 1000 ]]
    fi
```

- This second if statement checks now whether or not the value of \$1 is smaller than 1000 and only if this is also True we would execute the code afterwards

- Last but not least we need to add the then command, some code to execute and the fi command to close the if
- As you can see we now have multiple levels of indentation and a logical order in which the checks are executed

```
if [[ $1 -gt 100 ]]
then
    echo Hey that is a large number.
    if [[ $1 -lt 1000 ]]
    then
        echo Only three digits
    fi
fi
```


- Take your script from Exercise 22 and create nested if statements out of them by adding additional checks

1. <https://ryanstutorials.net/bash-scripting-tutorial/bash-script.php> (Last visited: 17.04.2023)
2. <https://phoenixnap.com/kb/bash-comment> (Last visited: 17.04.2023)
3. <https://ryanstutorials.net/bash-scripting-tutorial/bash-variables.php> (Last visted: 17.04.2023)
4. <https://ryanstutorials.net/bash-scripting-tutorial/bash-input.php> (Last visited: 17.04.2023)
5. <https://ryanstutorials.net/bash-scripting-tutorial/bash-if-statements.php> (Last visited: 17.04.2023)
6. <https://ryanstutorials.net/bash-scripting-tutorial/bash-arithmetic.php> (Last visited: 17.04.2023)
7. <https://ryanstutorials.net/bash-scripting-tutorial/bash-loops.php> (Last visited: 17.04.2023)
8. <https://ryanstutorials.net/bash-scripting-tutorial/bash-functions.php> (Last visited: 18.04.2023)